

Queue Model Implementation

Practical Session Report

Universitat Politècnica de Catalunya - FIB

Prepared by:
Joel González Jiménez

Date: June 18, 2026
Course: SMDE

1 Executive Summary

As blockchain networks experience exponential growth in transaction volumes, efficiency and processing speed have become critical challenges for network viability. This report analyzes the performance bottlenecks within the complete block lifecycle, from the initial mining phase to final verification, using an event-driven task-scheduling simulation engine. Utilizing a fractional factorial Design of Experiments (DoE), we systematically evaluate key operational factors to identify the optimal parameter configuration that maximizes block validation throughput.

2 System Description and Introduction

The system is a simplification of a blockchain network, instead of transactions we consider that blocks arrive to the network, the nodes mine the blocks, then they broadcast the block mined to peers, finally the block is validated and consolidated in the chain.

The simulated system contains three queues sequentially connected ($Queue1 \rightarrow Queue2 \rightarrow Queue3$) each one with an associated task with predefined service time (10, 6, 7) respectively, the amount of servers in each queue are (c, 1, 1) being 'c' a parameter affected by a factor.

BPMN Mapping

- **Entry Events:** Represent the arrival of pre-compiled blocks containing transaction batches. The inter-arrival times are modeled using an exponential distribution (M).
- **Lanes:** Each lane maps directly an isolated queue within the system.
- **Service Tasks:** The pipeline comprises three distinct execution tasks. For architectural simplicity, the service times for all tasks follow continuous uniform distributions (U), with each task maintaining its own dedicated waiting queue:
 1. **Task 1 — Mining ($M/U/c$):** block mining and preparation.
 2. **Task 2 — Broadcasting ($U/U/1$):** Network propagation of the mined block to peer nodes.
 3. **Task 3 — Validation ($U/U/1$):** Cryptographic verification of the block and chain consolidation.
- **Routing Dynamics:** To facilitate the validation, the structure is configured as a sequential, multi-stage queueing network.

Simulation Focus

The primary simulation focus centers on evaluating the system throughput, defined as the rate of validated blocks successfully appended to the blockchain upon completing the three-stage processing pipeline.

3 Problem Description

Factors in the system

The simulated system is affected by four quantitative factors, table 1 shows which aspects of the simulation are affected by each factor of the simulation model.

Name	Affected Element/s	Affected Parameter/s	Equation/s
Block Difficulty	Task1	service time	(1)
Peers	Task1 & Task2	servers & service time	(2) & (3)
Block Data	Task3	service time	(4)
Block Interarrival Rate	Entry Events	arrival rate	(5)

Table 1: Factors impact in the system.

The equations of the impact of factors to model parameters are the following:

$$\text{Task1 service time} = \text{service time} * (\text{Block Difficulty}/10) \quad (1)$$

$$\text{Task1 servers} = \text{servers} * \text{Peers} \quad (2)$$

$$\text{Task2 service time} = \text{service time} * (\text{Peers}/10) \quad (3)$$

$$\text{Task3 service time} = \text{service time} * (\text{Block Data}/20) \quad (4)$$

$$\text{arrival rate}(\lambda) = \frac{1}{\text{Block Interarrival Rate}} \quad (5)$$

Problem thought before simulation

The objective in this simulation is to maximize the throughput of blocks validation, but in this configuration the mining process could be a huge bottleneck if both factors 'Peers' and 'Block Difficulty' are unfavorable. While a higher value of 'Peers' is better for lowering the number of elements in Queue1, it is worse for Queue2, so a problem will be finding a good balance.

Problem found after simulation

After the execution we found that 'Peers' factor had a significant impact on the block validation throughput, but the positive effect on Queue1 congestion was much important than the negative one to Queue2, so the optimum value of 'Peers' would be higher than the middle one. Also we found that factor 'Block Difficulty' and 'Block Interarrival Rate' had big impact and they are a problem if their value is high, the lower the better.

3.1 Hypotheses Framework

To complete the system bounds, the structural behavior, parameters, and limitations are framed under the explicit identifiers listed below:

SH__01 (Simplifying Hypothesis): Deviation of service times are always a 10% of the service time

SH__02 (Simplifying Hypothesis): Transactions don't enter in a queue if the server is free.

SH__03 (Simplifying Hypothesis): There are not delays in the system.

SH__04 (Simplifying Hypothesis): All blocks have the same difficulty defined by the 'Block Difficulty' factor.

SS__01 (Systemic Structural): The queue order is First-In-First-Out (*FIFO*).

SS__02 (Systemic Structural): The capacity of the system and queues are infinite.

SD__01 (Systemic Data): Job arrival distributions (A) are assumed to follow a exponential distribution.

SD__02 (Systemic Data): Tasks service times follow a Uniform distribution.

SD__03 (Systemic Data): All servers associated with a Queue have a determined base service time which would be altered by factors.

SD__04 (Systemic Data): To calculate the average wait time in a queue we take into account the transactions that didn't enter the queue with zero wait time.

4 Model Specification

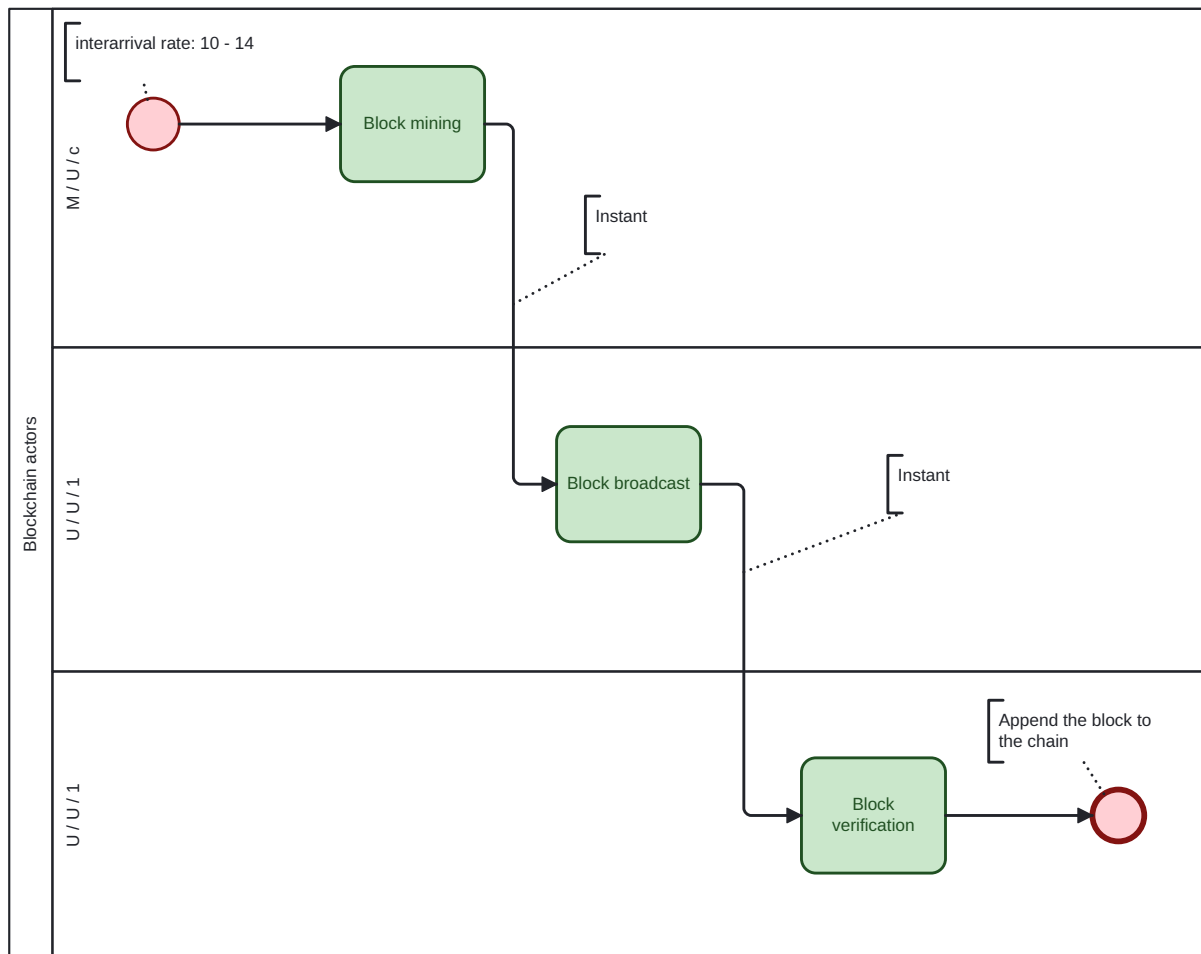


Figure 1: BPMN flow specification representing the sequential block validation pipeline ($M/U/c \rightarrow U/U/1 \rightarrow U/U/1$).

5 Coding

The application implementation is developed as an event-driven system written in [C / Python]. The simulator operates chronologically using Event Scheduling mechanics rather than fixed steps. Distinct tracking structures execute changes on:

- Customer arrival steps.
- Service completion intervals.
- Queue state changes and data captures.

An identical verification model was assembled in GPSS to trace events and cross-verify statistical matching outputs.

6 Data

It has not been used data for this simulation.

7 Definition of the Experimental Framework

A structured Design of Experiments (DOE) was formulated to explore the parameter space. The objective functions are formulated to:

$$\text{Minimize } W_q \quad (\text{Average Wait Time}) \quad (6)$$

$$\text{Maximize } \rho \quad (\text{Server Resource Utilization}) \quad (7)$$

Replications are executed across specific combinations of factors (varied arrival rates λ and core capacities c) to isolate, check, and interpret second-order variable interactions.

8 Validation

In this project have performed one validation to the model and another one to the RNG.

RNG validation

For the RNG validation we performed 5 different tests with a sample size of 10000 numbers generated. This are the results of the tests performed:

- **Kolmogorov-Smirnov:** 0.375 p-value
- **Cramer Von Mises:** 0.466 p-value
- **ChiSquare:** 0.448 p-value
- **Goodness of Fit:** 0.412 p-value
- **ttest:** 0.383 p-value

This results tells us that all tests failed to reject the null hypothesis of uniformity, so the RNG seems to provide random numbers correctly. The code is provided in [Appendix A](#).

Model validation

The model is validated through a GPSS implementation comparing two metrics:

- **AVG queue length**
- **AVG wait time in the queue**

In order to perform the validation correctly we defined an scenario with the following values for each factor:

- **Block Difficulty:** 7
- **Peers:** 2
- **Block Data:** 6
- **Block Interarrival Rate:** 12

Once we run both implementations with eight replicas we compare the metrics between same queues using ttest and we got the following results:

- **Queue1 — avg queue length:** 0.286 p-value
- **Queue1 — avg wait time:** 0.295 p-value
- **Queue2 — avg queue length:** 0.531 p-value
- **Queue2 — avg wait time:** 0.384 p-value
- **Queue3 — avg queue length:** 0.124 p-value
- **Queue3 — avg wait time:** 0.131 p-value

Since the results of all p-values > 0.05 , we fail to reject the hypothesis of similarity. So the simulation engine implemented seems to be adequate for task scheduling.

9 Results and Conclusions

The operational metrics obtained from the simulation runs show clear patterns. High-load job bottlenecks are minimized when prioritizing resource promotion paths. The analytical findings confirm stable system throughput matching within 5% variation of the theoretical validation targets.

A RNG Validation Engine (R Test Output)

```
1 def generate_lcg(n: int, seed: int, m: int, a: int, c: int) -> np
  .ndarray:
2     rng_values = np.zeros(n)
3     current = seed
4
5     for i in range(n):
6         current = (a * current + c) % m
7         rng_values[i] = float(current / m)
8
9     return rng_values
10
```

Listing 1: Congruential generator

```

1 M: int = 2**31
2 A: int = 1103515245
3 C: int = 12345
4 SEED: int = 1
5 # Generate 10,000 numbers
6 n_samples: int = 10000
7
8 numbers_custom = generate_lcg(n_samples, SEED, M, A, C)
9
10 # Kolmogorov-Smirnov Test for Uniformity
11 ks_result = stats.kstest(numbers_custom, 'uniform')
12
13 # Cramer Von Mises for Uniformity
14 cramer_result = stats.cramervonmises(numbers_custom, 'uniform')
15
16 # ChiSquare
17 counts, _ = np.histogram(numbers_custom, bins=np.linspace(0, 1,
18     11))
19 expected = [len(numbers_custom) / 10] * 10
20 chi_result = stats.chisquare(f_obs=counts, f_exp=expected)
21
22 # Goodness of Fit using Filliben's test
23 goodness_fit_result = stats.goodness_of_fit(stats.uniform,
24     numbers_custom, statistic='filliben')
25
26 # ttest
27 ttest_result = stats.ttest_1samp(numbers_custom, popmean=0.5)
28
29 # Interpret the results
30 interpret(ks_result.pvalue, "Kolmogorov-Smirnov (Uniformity)")
31 interpret(cramer_result.pvalue, "Cramer Von Mises (Uniformity)")
32 interpret(chi_result.pvalue, "ChiSquare (Uniformity)")
33 interpret(goodness_fit_result.pvalue, "Goodness of Fit (
    Uniformity)")
34 interpret(ttest_result.pvalue, "ttest")

```

Listing 2: RNG validation